Binary Search

Sean

October 10, 2026



Outline

- Why binary search works
- Binary search on sorted arrays
- First true / last true
- STL search and boundary operations
- Binary search on the answer



Introduction

- Binary search repeatedly cuts the search space in half

Introduction

- Binary search repeatedly cuts the search space in half
- It works when comparisons or a monotonic condition tell us which half to discard



Introduction

- Binary search repeatedly cuts the search space in half
- It works when comparisons or a monotonic condition tell us which half to discard
- In this lecture, we focus on discrete ordered spaces like indices, sorted values, and integer answers



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



Why Binary Search Works



When to Think Binary Search

- We need a fast way to find a position, value, or threshold



When to Think Binary Search

- We need a fast way to find a position, value, or threshold
- The valid answers form an ordered range



When to Think Binary Search

- We need a fast way to find a position, value, or threshold
- The valid answers form an ordered range
- We can tell whether the answer is to the left or right of a midpoint



When to Think Binary Search

- We need a fast way to find a position, value, or threshold
- The valid answers form an ordered range
- We can tell whether the answer is to the left or right of a midpoint
- Each check removes half of the remaining search space



Complexity

- One step reduces the interval size by about half



Complexity

- One step reduces the interval size by about half
- After k steps, only about $\frac{n}{2^k}$ candidates remain



Complexity

- One step reduces the interval size by about half
- After k steps, only about $\frac{n}{2^k}$ candidates remain
- That gives $\mathcal{O}(\log n)$ checks instead of $\mathcal{O}(n)$



Complexity

- One step reduces the interval size by about half
- After k steps, only about $\frac{n}{2^k}$ candidates remain
- That gives $\mathcal{O}(\log n)$ checks instead of $\mathcal{O}(n)$
- If one check costs $\mathcal{O}(C)$, total time is $\mathcal{O}(C \log n)$



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



Binary Search on Sorted Arrays



Sorted Array Search

- Start with an array sorted in increasing order



Sorted Array Search

- Start with an array sorted in increasing order
- Keep a search interval $[l, r]$



Sorted Array Search

- Start with an array sorted in increasing order
- Keep a search interval $[l, r]$
- Look at the middle position



Sorted Array Search

- Start with an array sorted in increasing order
- Keep a search interval $[l, r]$
- Look at the middle position
- If it is too small, discard the left half; if too large, discard the right half



Sorted Array Search

- Start with an array sorted in increasing order
- Keep a search interval $[l, r]$
- Look at the middle position
- If it is too small, discard the left half; if too large, discard the right half
- If it equals the target, return its index



Exact Match Template

```cpp
int find_index(const vector<int>& a, int x) {
  int l = 0, r = (int)a.size() - 1; // empty: r = -1
  while (l <= r) {
    int mid = l + (r - l) / 2;
    if (a[mid] == x) return mid; // any matching index
    if (a[mid] < x) l = mid + 1; // discard mid too
    else r = mid - 1;
  }
  return -1; // not found; do not use as an index
}
```

C++17; snippets assume `using namespace std;` and the headers `<algorithm>`, `<vector>`, and `<iostream>`.



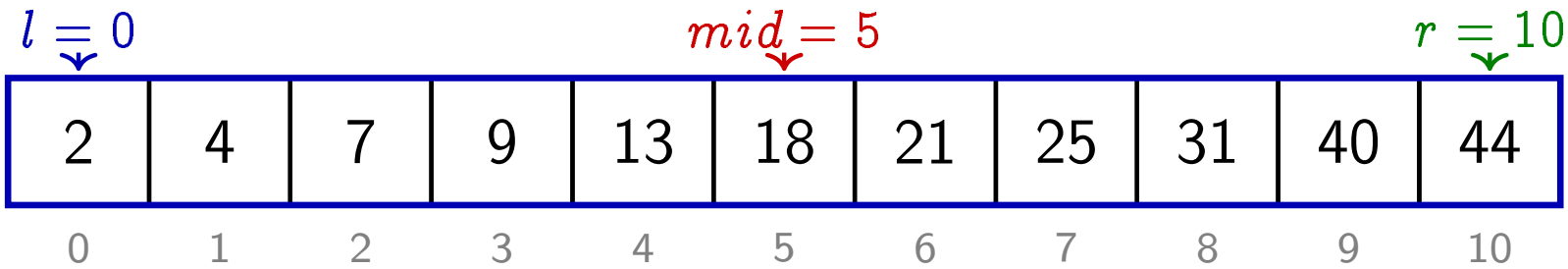
Exact search in code

```cpp
vector<int> a = {2, 5, 5, 8}; // sorted; zero-based
cout << find_index(a, 8);     // 3
cout << find_index(a, 6);     // -1: absent
int i = find_index(a, 5);     // 1 with this template
if (i != -1) cout << a[i];    // 5; check before access
cout << find_index(a, 1);     // -1: below every value
cout << find_index(a, 9);     // -1: above every value
cout << find_index(vector<int>{}, 5); // -1: empty
// Exact search may return ANY copy of a duplicate.
// Use a boundary search when the first copy matters.
```



Trace on 11 Values

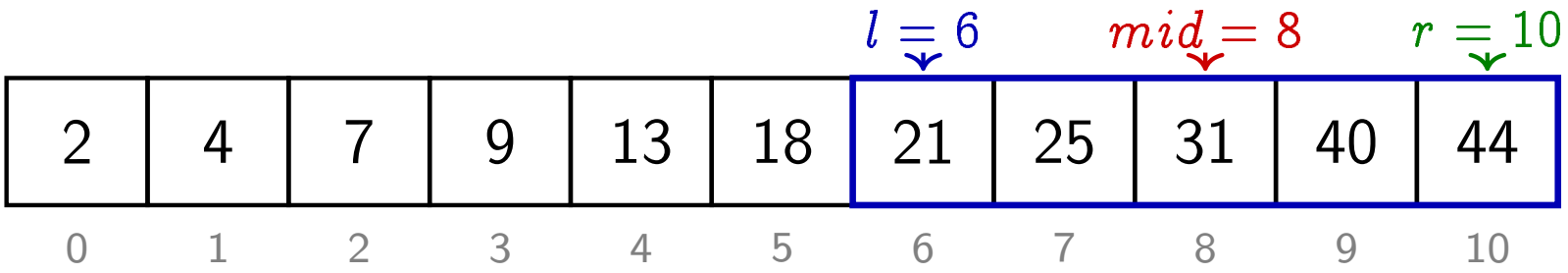
Find 25 in the sorted array below.



$a[mid] = 18 < 25$, so set $l = 6$

Trace on 11 Values

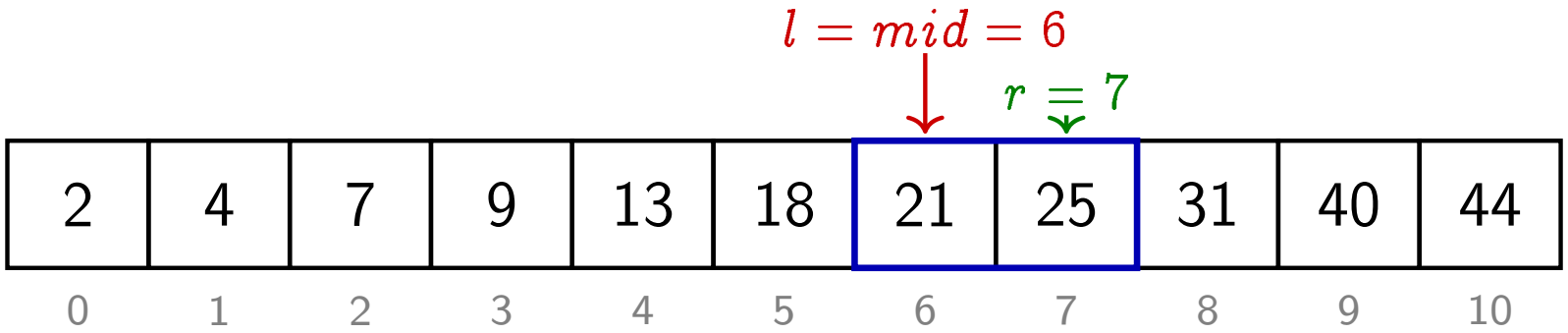
Find 25 in the sorted array below.



$a[mid] = 31 > 25$, so set $r = 7$

Trace on 11 Values

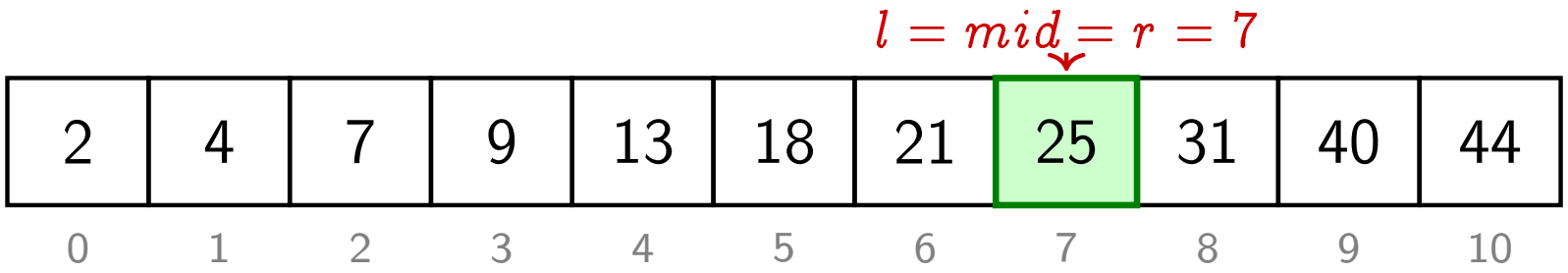
Find 25 in the sorted array below.



$a[mid] = 21 < 25$, so set $l = 7$

Trace on 11 Values

Find 25 in the sorted array below.



$a[mid] = 25$, so we found the answer

Common Invariants

- An invariant is a fact that stays true every time the loop runs



Common Invariants

- An invariant is a fact that stays true every time the loop runs
- For example: “if the answer exists, it is still inside the current search interval”



Common Invariants

- An invariant is a fact that stays true every time the loop runs
- For example: “if the answer exists, it is still inside the current search interval”
- Make sure every update keeps that fact true and also shrinks the interval

Common Invariants

- An invariant is a fact that stays true every time the loop runs
- For example: “if the answer exists, it is still inside the current search interval”
- Make sure every update keeps that fact true and also shrinks the interval
- Most bugs come from breaking the invariant or mixing interval conventions



Template Conventions

- Competitive programmers use several valid binary search templates



Template Conventions

- Competitive programmers use several valid binary search templates
- Some use closed intervals like $[l, r]$, and others use half-open intervals like $[l, r)$



Template Conventions

- Competitive programmers use several valid binary search templates
- Some use closed intervals like $[l, r]$, and others use half-open intervals like $[l, r)$
- Some exact-search templates use `while (l <= r)`, while many boundary-search templates use `while (l < r)`



Template Conventions

- Competitive programmers use several valid binary search templates
- Some use closed intervals like $[l, r]$, and others use half-open intervals like $[l, r)$
- Some exact-search templates use `while (l <= r)`, while many boundary-search templates use `while (l < r)`
- Our templates use $[l, r]$, `l <= r`, and discard `mid` with `mid + 1` or `mid - 1`



Midpoint Choices

- Many people write the midpoint as `(l + r) / 2`



Midpoint Choices

- Many people write the midpoint as `(l + r) / 2`
- For our nonnegative indices and answer ranges, `l + (r - l) / 2` avoids overflow in `l + r`



Midpoint Choices

- Many people write the midpoint as `(l + r) / 2`
- For our nonnegative indices and answer ranges, `l + (r - l) / 2` avoids overflow in `l + r`
- Some templates use the lower midpoint and others use the upper midpoint



Midpoint Choices

- Many people write the midpoint as `(l + r) / 2`
- For our nonnegative indices and answer ranges, `l + (r - l) / 2` avoids overflow in `l + r`
- Some templates use the lower midpoint and others use the upper midpoint
- The midpoint formula must match your update rules, or the loop can get stuck



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



First True / Last True



Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true



Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true
- Think of a boolean array of the form false, false, ..., true, true



Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true
- Think of a boolean array of the form false, false, ..., true, true
- Binary search finds the boundary between the two regions



Boundary Search

- Often we do not want an exact value; we want the first place a condition becomes true
- Think of a boolean array of the form false, false, ..., true, true
- Binary search finds the boundary between the two regions
- This is the pattern behind `lower_bound` and `upper_bound`



First True Template

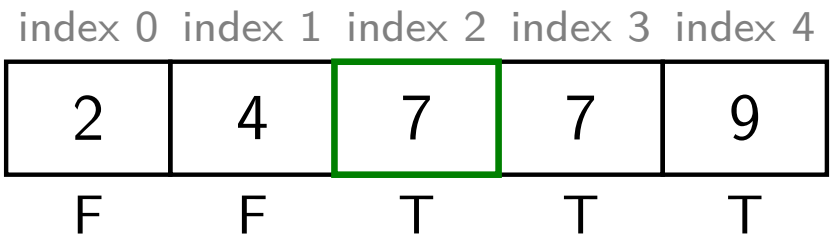
```cpp
template<class Predicate>
int first_true(int n, Predicate good) {
  int l = 0, r = n - 1, ans = n; // n means no true
  while (l <= r) {
    int mid = l + (r - l) / 2;
    if (good(mid)) {
      ans = mid;   // save this valid candidate
      r = mid - 1; // look for an earlier true
    } else l = mid + 1;
  }
  return ans; // also returns n when n == 0
}
```

Requires `false, ..., false, true, ..., true` on $[0, n)$. The saved candidate lets us discard `mid`.



Trace: finding the first true

Find the first index with $a[i] \geq 7$.



Interval	mid	**Check**	**Update**
$[0, 4]$	2	true	Save $ans = 2$; set $r = 1$
$[0, 1]$	0	false	Set $l = 1$
$[1, 1]$	1	false	Set $l = 2$

Stop at $l = 2 > r = 1$. The saved answer is 2, the first copy of 7.

Last True Template

```cpp
template<class Predicate>
int last_true(int n, Predicate good) {
  int l = 0, r = n - 1, ans = -1; // no true yet
  while (l <= r) {
    int mid = l + (r - l) / 2;
    if (good(mid)) {
      ans = mid;   // save this valid candidate
      l = mid + 1; // look for a later true
    } else r = mid - 1;
  }
  return ans; // also returns -1 when n == 0
}
```

Requires `true, ..., true, false, ..., false` on $[0, n)$.



Boundary searches in code

```cpp
vector<int> a = {2, 5, 5, 5, 8};
int n = (int)a.size();
int first = first_true(n, [&](int i) {
  return a[i] >= 5; // F T T T T
}); // first = 1
int last = last_true(n, [&](int i) {
  return a[i] <= 5; // T T T T F
}); // last = 3
if (first != n) cout << a[first]; // 5
if (last != -1) cout << a[last];  // 5
// first_true: all false -> n; all true -> 0.
// last_true: all false -> -1; all true -> n - 1.
// With n == 0, neither function calls good().
```



Typical Uses

- First element greater than or equal to x



Typical Uses

- First element greater than or equal to x
- First element strictly greater than x



Typical Uses

- First element greater than or equal to x
- First element strictly greater than x
- Last position where a predicate is true



Typical Uses

- First element greater than or equal to x
- First element strictly greater than x
- Last position where a predicate is true
- Counting values in a sorted range using two boundaries



lower_bound and upper_bound

```cpp
vector<int> a = {2, 5, 5, 5, 8}; // sorted
auto lo = lower_bound(a.begin(), a.end(), 5);
auto hi = upper_bound(a.begin(), a.end(), 5);
cout << lo - a.begin(); // 1: first value >= 5
cout << hi - a.begin(); // 4: first value > 5
cout << hi - lo;        // 3: number of copies of 5
if (lo != a.end()) cout << *lo; // 5
if (hi != a.end()) cout << *hi; // 8
// Return iterators; end() means no qualifying value.
// Sorted vectors give O(log n) time per search.
```



Missing values and end()

```
1  vector<int> a = {2, 5, 5, 5, 8};
2  int x = 6; // absent, but lower_bound is not end()
3  auto it = lower_bound(a.begin(), a.end(), x);
4  if (it != a.end() && *it == x) cout << "found";
5  else cout << "absent";        // absent; it points to 8
6  auto lo = lower_bound(a.begin(), a.end(), 9);
7  cout << lo - a.begin();       // 5: index == a.size()
8  if (lo != a.end()) cout << *lo; // skipped; no value
9  vector<int> empty;
10 auto e = lower_bound(empty.begin(), empty.end(), 5);
11 cout << (e == empty.end());  // true
12 // Check end() BEFORE dereferencing the iterator.
```



Counting a sorted value range

```cpp
vector<int> a = {2, 5, 5, 5, 8};
int L = 3, R = 8; // query values, not array indices
auto left = lower_bound(a.begin(), a.end(), L);
auto right = upper_bound(a.begin(), a.end(), R);
cout << right - left; // 4: values in [3, 8]
auto lo = lower_bound(a.begin(), a.end(), 5);
auto hi = upper_bound(a.begin(), a.end(), 5);
cout << lo - a.begin(); // 1: values < 5
cout << hi - a.begin(); // 4: values <= 5
cout << a.end() - hi;   // 1: values > 5
// Require L <= R. right is an exclusive endpoint.
// Sort once: O(n log n); each query: O(log n).
```



Other STL search operations

```cpp
vector<int> a = {2, 5, 5, 5, 8};
cout << binary_search(a.begin(), a.end(), 5); // true
cout << binary_search(a.begin(), a.end(), 6); // false
auto [first, last] = equal_range(a.begin(), a.end(), 5);
cout << first - a.begin(); // 1
cout << last - a.begin();  // 4: exclusive endpoint
cout << last - first;      // 3: matching values
auto [lo, hi] = equal_range(a.begin(), a.end(), 6);
cout << hi - lo;           // 0: absent
// binary_search returns bool, not an index.
// equal_range returns {lower_bound, upper_bound}.
// All four STL searches here use the same sorted order.
```



Removing duplicates before searching

```cpp
vector<int> a = {5, 2, 5, 8, 5};
sort(a.begin(), a.end());    // {2, 5, 5, 5, 8}
auto new_end = unique(a.begin(), a.end());
cout << a.size();            // 5: size has NOT changed
cout << new_end - a.begin(); // 3: logical unique length
a.erase(new_end, a.end());  // now a = {2, 5, 8}
auto it = lower_bound(a.begin(), a.end(), 5);
cout << it - a.begin();      // 1: rank among distinct values
// Idiom: a.erase(unique(a.begin(), a.end()), a.end());
// unique removes only consecutive duplicates; sort first.
// Do not deduplicate if multiplicities matter.
```



1 Why Binary Search Works

2 Binary Search on Sorted Arrays

3 First True / Last True

4 Binary Search on the Answer



Binary Search on the Answer



Searching the Answer Space

- Sometimes the array is not the search space



Searching the Answer Space

- Sometimes the array is not the search space
- Instead, guess an answer and check whether it is feasible



Searching the Answer Space

- Sometimes the array is not the search space
- Instead, guess an answer and check whether it is feasible
- For a minimum answer, the monotonic check is false then true; for a maximum, it is true then false



Searching the Answer Space

- Sometimes the array is not the search space
- Instead, guess an answer and check whether it is feasible
- For a minimum answer, the monotonic check is false then true; for a maximum, it is true then false
- Then binary search finds the smallest or largest feasible answer



Example: Factory Machines

CSES: Factory Machines

Given n machine times k_i, find the smallest time needed to produce at least t products. Here $n \leq 2 \times 10^5$ and $1 \leq t, k_i \leq 10^9$.



Example: Factory Machines

CSES: Factory Machines

Given n machine times k_i, find the smallest time needed to produce at least t products. Here $n \leq 2 \times 10^5$ and $1 \leq t, k_i \leq 10^9$.

- In s seconds, machine i makes $\lfloor s/k_i \rfloor$ products



Example: Factory Machines

CSES: Factory Machines

Given n machine times k_i, find the smallest time needed to produce at least t products. Here $n \leq 2 \times 10^5$ and $1 \leq t, k_i \leq 10^9$.

- In s seconds, machine i makes $\lfloor s/k_i \rfloor$ products
- `good(s)` asks whether $\sum_i \lfloor s/k_i \rfloor \geq t$



Example: Factory Machines

CSES: Factory Machines

Given n machine times k_i, find the smallest time needed to produce at least t products. Here $n \leq 2 \times 10^5$ and $1 \leq t, k_i \leq 10^9$.

- In s seconds, machine i makes $\lfloor s/k_i \rfloor$ products
- `good(s)` asks whether $\sum_i \lfloor s/k_i \rfloor \geq t$
- More time cannot reduce production: false then true



Example: Factory Machines

CSES: Factory Machines

Given n machine times k_i, find the smallest time needed to produce at least t products. Here $n \leq 2 \times 10^5$ and $1 \leq t, k_i \leq 10^9$.

- In s seconds, machine i makes $\lfloor s/k_i \rfloor$ products
- `good(s)` asks whether $\sum_i \lfloor s/k_i \rfloor \geq t$
- More time cannot reduce production: false then true
- Save a feasible time, then search for an earlier one

Trace: Factory Machines

Machine times $[3, 2, 5]$, target 7. The fastest machine alone finishes in $2 \cdot 7 = 14$ seconds.

Interval	mid	**Products**	**Update**
$[0, 14]$	7	$2 + 3 + 1 = 6$	Not enough: $l = 8$
$[8, 14]$	11	$3 + 5 + 2 = 10$	Save 11; $r = 10$
$[8, 10]$	9	$3 + 4 + 1 = 8$	Save 9; $r = 8$
$[8, 8]$	8	$2 + 4 + 1 = 7$	Save 8; $r = 7$

The boundary

7 seconds is too short; 8 seconds works. The minimum feasible time is 8.

Factory Machines: safe bounds

```cpp
int n;
long long target;
cin >> n >> target;
vector<long long> machine(n);
for (long long& k : machine) cin >> k;
long long fastest = *min_element(
    machine.begin(), machine.end());
long long l = 0, r = fastest * target;
long long ans = r; // fastest machine alone is enough
// n >= 1 and every machine time is positive.
// Both factors are long long; r <= 10^18 here.
```

The upper bound must be feasible. Use the following snippets in the same scope after reading the input.



Factory Machines: a safe check

```cpp
auto good = [&](long long seconds) {
  long long made = 0;
  for (long long k : machine) {
    long long add = seconds / k;
    if (add >= target - made) return true;
    made += add; // still below target
  }
  return false; // all machines together are too slow
};
// We only need to know whether production reaches target.
// Compare before adding; never accumulate a huge total.
// One check scans at most n machines: O(n).
```



Factory Machines: search and output

```cpp
while (l <= r) {
  long long mid = l + (r - l) / 2;
  if (good(mid)) {
    ans = mid;   // feasible; keep this candidate
    r = mid - 1; // try an earlier finish time
  } else {
    l = mid + 1; // need more time
  }
}
cout << ans << '\n'; // times {3, 2, 5}, target 7 -> 8
// U = fastest * target; total time is O(n log(U + 1)).
// At most about 60 checks when U <= 10^18.
```



Choosing a search operation

Need	Use
Does x exist?	`binary_search`
Any index of x	Exact search; check for -1
First value $\geq x$ / $> x$	`lower_bound` / `upper_bound`
All copies of x	`equal_range`; count the iterator gap
Minimum feasible answer	First true: false $\rightarrow$ true
Maximum feasible answer	Last true: true $\rightarrow$ false

STL examples above use sorted vectors. Answer-space searches need a monotonic predicate and justified bounds.

Boundary and STL mistakes

- Treating `lower_bound(x)` as proof that x exists: check both `end()` and equality



Boundary and STL mistakes

- Treating `lower_bound(x)` as proof that x exists: check both `end()` and equality
- Reading `a[n]`, `a[-1]`, or `*end()` when no answer exists



Boundary and STL mistakes

- Treating `lower_bound(x)` as proof that x exists: check both `end()` and equality
- Reading `a[n]`, `a[-1]`, or `*end()` when no answer exists
- Assuming exact search returns the first duplicate



Boundary and STL mistakes

- Treating `lower_bound(x)` as proof that x exists: check both `end()` and equality
- Reading `a[n]`, `a[-1]`, or `*end()` when no answer exists
- Assuming exact search returns the first duplicate
- For `set`/`map`, use their member `lower_bound`; the generic algorithm can take linear iterator steps



Common Mistakes

- Overflow in bounds or products, even when the midpoint formula is safe



Common Mistakes

- Overflow in bounds or products, even when the midpoint formula is safe
- Infinite loops from not shrinking the interval



Common Mistakes

- Overflow in bounds or products, even when the midpoint formula is safe
- Infinite loops from not shrinking the interval
- Using binary search on a non-monotonic check



Common Mistakes

- Overflow in bounds or products, even when the midpoint formula is safe
- Infinite loops from not shrinking the interval
- Using binary search on a non-monotonic check
- Forgetting empty input, no feasible answer, or every candidate being feasible



Takeaways

- Binary search needs order plus a clear invariant
- Exact search and boundary search are closely related
- `lower_bound` and `upper_bound` are standard tools
- Answer-space binary search depends on a monotonic check

5 Appendix



Appendix



Appendix: Invariant

- An invariant is a fact that stays true during the whole loop
- Example: “if the answer exists, it is still inside the current interval”
- Every update to `l` or `r` should keep that fact true

Appendix: Monotonic

- Monotonic means the check changes in only one direction
- Typical pattern: false, false, false, true, true, true
- That creates one clean boundary, which is why binary search works



Appendix: Discrete

- Discrete means the search space is made of separate countable choices
- Examples here: array indices, sorted values, and integer answers
- That is different from searching over all real numbers



Appendix: Half-Open Interval

- The interval $[l, r)$ includes `l` but does not include `r`
- So it contains positions `l, l + 1, ..., r - 1`
- Many binary search templates use this style because the right endpoint stays exclusive



Appendix: Overflow

- Overflow means a value goes past the range its integer type can store
- If `l` and `r` are large, `l + r` can overflow
- That is why `l + (r - l) / 2` is safer than `(l + r) / 2`



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

